22360859088
KORAY GARİP

BURSA TEKNİK ÜNİVERSİTESİ

MÜHENDİSLİK VE DOĞA BİLİMLERİ FAKÜLTESİ

BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ

Node.JS ile Web Programlama

LAB_8

22360859088
KORAY GARİP

1.Express.js ile Web Sunucu Oluşturma

Bu aşamada Node.js üzerinde web sunucu geliştirmek için Express.js kütüphanesi kullanılmıştır. Express, Node.js ile server kurmayı kolaylaştıran popüler bir npm modülüdür. Bu amaçla yeni bir webServer klasörü oluşturulmuş, npm init ile proje başlatılmış ve npm i express komutu ile Express kurulmuştur.

Proje düzenli ilerlemesi için src klasörü oluşturulmuş ve ana dosya olarak app.js tanımlanmıştır. Express kullanımı için öncelikle:

```
const express = require("express")
const app = express()
```

şeklinde bir uygulama nesnesi oluşturulmuştur.

Daha sonra app.get() fonksiyonu kullanılarak belirli URL'lere karşılık çalışacak fonksiyonlar tanımlanmıştır. Bu fonksiyonlar req (request) ve res (response) parametrelerini alır ve res.send() ile kullanıcıya çıktı gönderilir. Gönderilen veri string olabileceği gibi HTML veya JSON formatında da olabilir.

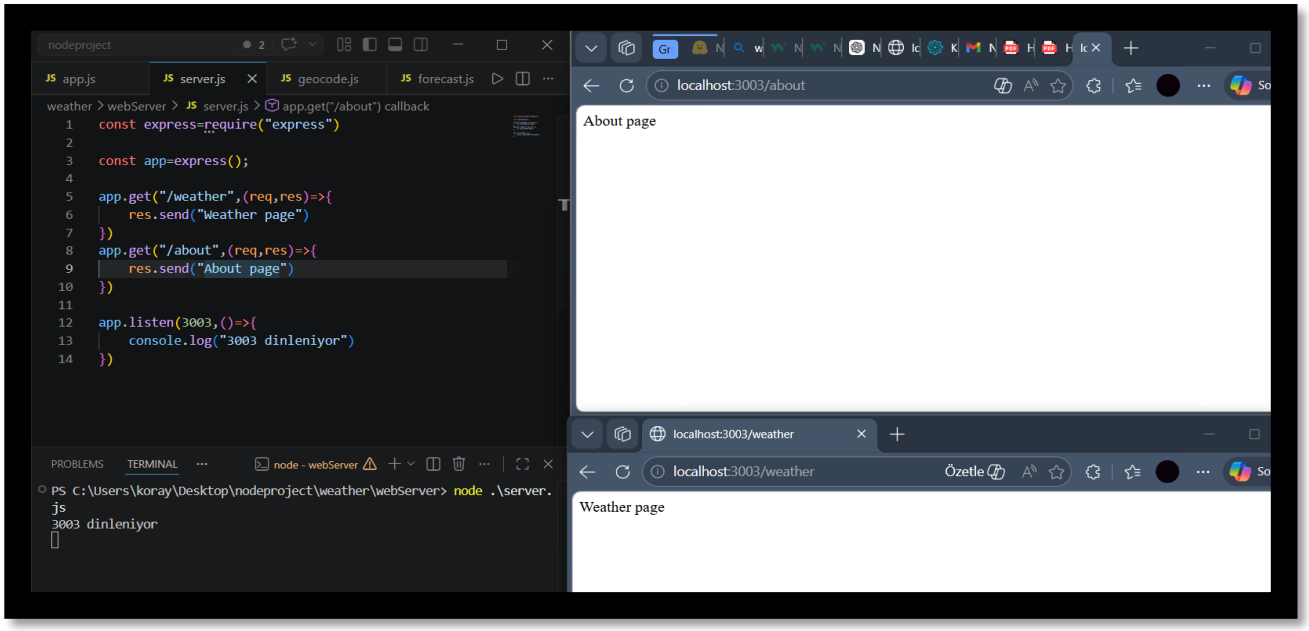
Server'ın çalışması için:

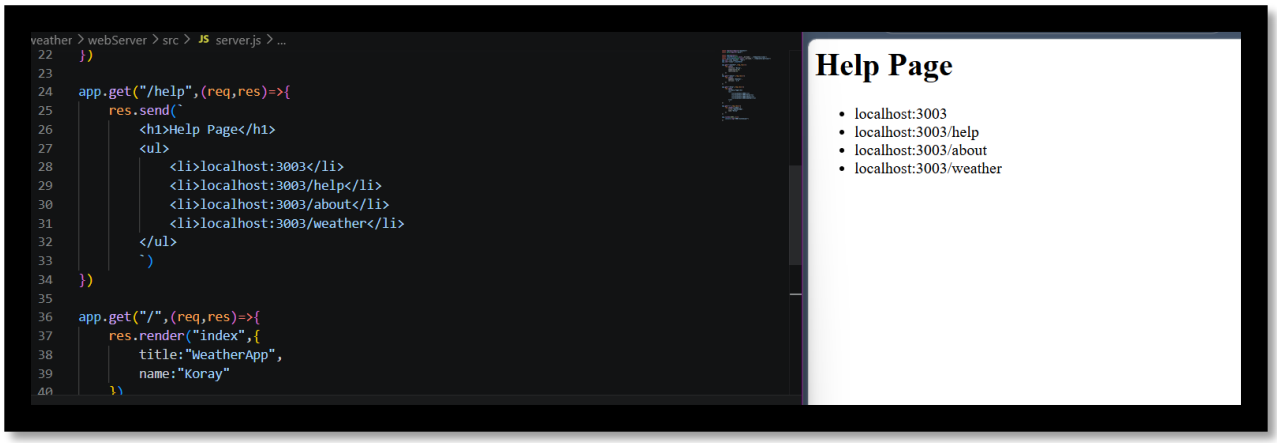
```
app.listen(3003,()=>{
    console.log("3003 dinleniyor")
})
```

kullanılmış ve uygulama localhost:3003 üzerinden test edilmiştir.

Ayrıca uygulama kapsamında /about ve /weather gibi farklı sayfalar oluşturularak, her bir sayfa için ayrı içerikler gösterilmiştir.

Örnek1:





2. Static Yapı ve Dosya Yönetimi

İlerleyen aşamada, HTML içeriklerinin doğrudan JavaScript içinde string olarak yazılmasının karmaşıklaştığı görülmüş ve bunun yerine static , yapı kullanımı öğrenilmiştir. Bu amaçla proje içine public klasörü oluşturulmuş ve HTML dosyaları (index.html, about.html, help.html) bu klasör içine yerleştirilmiştir.

Node.js'te dosya yollarını doğru yönetmek için:

- __dirname → bulunduğumuz klasörün yolu
- __filename → dosyanın tam yolu

kavramları incelenmiştir.

Ayrıca path işlemlerini kolaylaştırmak için Node.js'in dahili path modülü kullanılmıştır:

```
const path=require("path")

const viewsPath=path.join(__dirname,'../templates/views')
const partialPath=path.join(__dirname,"../templates/partials")
app.set("view engine", "hbs")
app.set("views",viewsPath)
```

3.Dinamik Sayfa Yapısı

Dinamik sayfa yapısı oluşturmak için Express ile birlikte hbs (Handlebars) kütüphanesi kullanılmıştır. Bu amaçla proje içerisine npm i hbs komutu ile gerekli paket kurulmuştur. Statik HTML sayfaları yerine, sayfa uzantıları .html yerine .hbs olarak değiştirilerek dinamik hale getirilmiştir.

Dinamik sayfalara veri göndermek için Express içerisinde res.render() fonksiyonu kullanılmıştır. Bu fonksiyon sayesinde backend tarafında oluşturulan veriler template dosyasına aktarılabilmektedir. .hbs dosyalarında ise gönderilen veriler, key değerleri kullanılarak doğrudan ekrana yazdırılabilmektedir.

Bu yapı sayesinde sayfalar sabit olmaktan çıkarılmış, kullanıcıdan gelen veya API'den alınan veriler doğrultusunda dinamik olarak içerik üreten bir sistem oluşturulmuştur. Böylece modern web uygulamalarında kullanılan template mantığı uygulamalı olarak öğrenilmiştir.

weather > webServer > src > server.js > ...

```
app.get("/",(req,res)=>{
    res.render("index",{
        title:"WeatherApp",
        name:"Koray"
    })
})

app.listen(3003,()=>{
    console.log("3003 dinleniyor")
})
```

weather > webServer > templates > views > index.hbs > ...

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width,
    initial-scale=1.0">
    <link rel="stylesheet" href="css/style.css">
    <title>Weather App</title>
</head>
<body>
    <h1>{{title}}</h1>
    <p>HBS DİNAMİK SAYFA</p>
    <p>Bu sayfa {{name}} tarafından hazırlandı</p>
</body>
</html>
```

Örnek 2:

weather > webServer > src > server.js > ...

```
const express=require("express")
const path=require("path")

const app=express();
const viewsPath=path.join(__dirname,'../templates/views')
const partialPath=path.join(__dirname,"../templates/partials")
app.set("view engine", "hbs")
app.set("views",viewsPath)

app.get("/weather",(req,res)=>{
    res.send({
        location:'Bursa',
        temperature:20,
        feelslike:22
    })
})
app.get("/about",(req,res)=>{
    res.send({
```

PROBLEMS OUTPUT DEBUG CONSOLE PORTS TERMINAL

```
PS C:\Users\koray\Desktop\nodeproject\weather\webServer\s
rc> nodemon server.js -e js,hbs
    at Layer.handleRequest (C:\Users\koray\Desktop\nodeproject\weather\webServer\node_modu
les\router\lib\layer.js:152:17)
    at C:\Users\koray\Desktop\nodeproject\weather\webServer\node_modules\router\index.js:2
95:15
    at processParams (C:\Users\koray\Desktop\nodeproject\weather\webServer\node_modules\ro
uter\index.js:582:12)
PS C:\Users\koray\Desktop\nodeproject\weather\webServer\src> nodemon server.js -e js,hbs
[nodemon] 3.1.14
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,hbs
[nodemon] starting `node server.js`
3003 dinleniyor
```

powers... PowerShell ... 1 Hidden T...

WeatherApp

HBS DİNAMİK SAYFA

Bu sayfa Koray tarafından hazırlandı